SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Pattern Recognition and Text Processing

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Pattern Recognition and Text Processing
Weightage:	40% of CIA		
CO5 Statement:	Analyse regular expression patterns. (BL-3)		
Student Name:	Abinesh H	**Reg. No.:**	192424387
Section / Batch:	B.Tech AIDS / 3rd Year	**Date:**	21/07/2026

Code of Conduct

I ___Abinesh H (192424387)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Abinesh H_____

Instructions

- Show all intermediate steps, calculations, state transitions, and reasoning wherever applicable.
- Use only the Python re (Regular Expressions) module to solve the given problems.
- Clearly mention the Regular Expression (Regex) pattern used before writing the corresponding Python program.
- Display the required output for each question, including extracted values, validation results, counts, and positions wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Generation	Pattern Matching Information Extraction	1
Search for a String	Keyword Search and Text Analysis	2
Split the documentation into sentences and words	Text Tokenization and Sentence Segmentation	3
Email and Strong Password Validation	Data Validation and Format Verification	4
Compile Once, Validate Many	Pattern Reusability and Performance Optimization	5

Annexure A – Pattern Recognition and Text Processing

Section 1: Regular Expression Pattern Generation

Student Details:
Name: John Doe
Email: john.doe123@gmail.com
Mobile: +91-9876543210
Password: P@ssw0rd123
Date of Birth: 15/08/2004
Register Number: 23AIML1056
Department: Artificial Intelligence and Machine Learning
For the given student details formulate Regex patterns for the following:
Email ID
Mobile Number
Strong Password
Date of Birth (DD/MM/YYYY)
Register Number

Section 2: Search for a String

Artificial Intelligence (AI) is transforming industries across the world. AI is used in healthcare to assist doctors in diagnosis, in banking to detect fraud, and in education to provide personalized learning experiences. Many companies invest heavily in AI research because AI improves efficiency and enables intelligent decision-making. As AI continues to evolve, professionals with AI skills are in high demand.
Write a Python program using the re module to perform the following operations for the word "AI":

1. Find the first occurrence of the word "AI" using re.search().
2. Display the starting position of the first occurrence.
3. Display the ending position of the first occurrence.
4. Display the total number of times the word "AI" appears in the passage.
5. Print an appropriate message if the word is not found.

Section 3: Split the documentation into sentences and words

Refer the previous passage and Write a Python program using the re.split() method to perform the following operations:

- Split the passage into sentences using appropriate punctuation (., ?, !) as delimiters.
- Count and display the total number of sentences.
- Split the passage into words by considering one or more whitespace characters as delimiters.
- Count and display the total number of words.
- Display the list of extracted sentences and words.

Section 4: Email and Strong Password Validation

A website requires users to register by providing an Email ID and a Strong Password. Write a Python program using the re module to validate the following inputs.
Validation Criteria:
Email ID

- Must begin with one or more letters or digits.
- May contain ., _, %, +, or - before the @ symbol.
- Must contain a valid domain name.
- Must end with a valid domain extension such as .com, .org, .edu, or .in.

Strong Password

- Must contain at least 8 characters.
- Must contain at least one uppercase letter.
- Must contain at least one lowercase letter.
- Must contain at least one digit.
- Must contain at least one special character from @, #, $, %, &, !, or *.
- Must not contain any whitespace characters.

Section 5: Compile Once, Validate Many

A company receives 1,000 email IDs from its customers and needs to validate each one efficiently. Instead of compiling the same regular expression for every email, write a Python program using the re.compile() method to compile the pattern once and reuse it to validate all the email IDs.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Q. No. / Criteria	Conceptual Understanding	Accuracy of Answer	Application of Knowledge	Completeness of Response	Clarity & Presentation	Sub. Total	Total %
1	3	3	3	4	4	17	81%
2	3	3	3	4	4	17	
3	3	3	3	4	4	17	
4	3	3	3	3	3	15	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Signature: Dr. Kumaragurubaran T	Signature: _____	Signature: _____
Date: 21/07/2026	Date: _____	Date: _____

Section 1: Regular Expression Pattern Generation

Student Details:

Name: John Doe

Email: ^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$

Mobile : ^\+91-[6-9]\d{9}$

Password : ^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,}$

Date of Birth (DD/MM/YYYY) : `^(0[1-9]

Register Number : ^\d{2}[A-Z]{4}\d{4}$

Section 2: Search for a String

```python
import re

text = """Artificial Intelligence (AI) is transforming industries across the wor

word = "AI"

# 1. Find the first occurrence
match = re.search(r'\b' + word + r'\b', text)

if match:
    # 2. Starting position
    print("First occurrence found.")
    print("Starting Position:", match.start())

    # 3. Ending position
    print("Ending Position:", match.end())

    # 4. Total occurrences
    count = len(re.findall(r'\b' + word + r'\b', text))
    print("Total Occurrences:", count)
else:
    # 5. Word not found
    print("The word 'AI' was not found in the passage.")
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
=== RESTART: C:/Users/Abinesh/AppData/Local/Programs/Python/Python311/at1.py ===
First occurrence found.
Starting Position: 25
Ending Position: 27
Total Occurrences: 6
>>>
```

Section 3: Split the documentation into sentences and words

```python
import re

text = """Artificial Intelligence (AI) is transforming industries across the wor

# Split into sentences
sentences = [s.strip() for s in re.split(r'[.!?]+', text) if s.strip()]

# Split into words
words = re.split(r'\s+', text.strip())

# Display sentences
print("Sentences:")
for i, sentence in enumerate(sentences, 1):
    print(f"{i}. {sentence}")

print("\nTotal Number of Sentences:", len(sentences))

# Display words
print("\nWords:")
print(words)

print("\nTotal Number of Words:", len(words))
```

```
== RESTART: C:/Users/Abinesh/AppData/Local/Programs/Python/Python311/at1.1.py ==
Sentences:
1. Artificial Intelligence (AI) is transforming industries across the world
2. AI is used in healthcare to assist doctors in diagnosis, in banking to detect
 fraud, and in education to provide personalized learning experiences
3. Many companies invest heavily in AI research because AI improves efficiency a
nd enables intelligent decision-making
4. As AI continues to evolve, professionals with AI skills are in high demand

Total Number of Sentences: 4

Words:
['Artificial', 'Intelligence', '(AI)', 'is', 'transforming', 'industries', 'acro
ss', 'the', 'world.', 'AI', 'is', 'used', 'in', 'healthcare', 'to', 'assist', 'd
octors', 'in', 'diagnosis,', 'in', 'banking', 'to', 'detect', 'fraud,', 'and', '
in', 'education', 'to', 'provide', 'personalized', 'learning', 'experiences.', '
Many', 'companies', 'invest', 'heavily', 'in', 'AI', 'research', 'because', 'AI'
, 'improves', 'efficiency', 'and', 'enables', 'intelligent', 'decision-making.',
 'As', 'AI', 'continues', 'to', 'evolve,', 'professionals', 'with', 'AI', 'skill
s', 'are', 'in', 'high', 'demand.']

Total Number of Words: 60
>>>
```

Section 4: Email and Strong Password Validation

at1.2.py - C:/Users/Abinesh/AppData/Local/Programs/Python/Python311/at1.2.py (3.11.9)

File Edit Format Run Options Window Help

```
import re

# Get user input
email = input("Enter Email ID: ")
password = input("Enter Password: ")

# Regex patterns
email_pattern = r'^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
password_pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$%&!*])[A-Za-z\d@#$%&!*]{8,}$'

# Validate Email
if re.fullmatch(email_pattern, email):
    print("Valid Email ID")
else:
    print("Invalid Email ID")

# Validate Password
if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Invalid Password")
```

Ln: 22 Col: 0

```
>>>
== RESTART: C:/Users/Abinesh/AppData/Local/Programs/Python/Python311/at1.2.py ==
Enter Email ID: abineshabi3112@gamil.com
Enter Password: Heramabi@2210
Valid Email ID
Strong Password
>>>
== RESTART: C:/Users/Abinesh/AppData/Local/Programs/Python/Python311/at1.2.py ==
Enter Email ID: _abine$h?gmail-in
Enter Password: ;vjrhx(ln
Invalid Email ID
Invalid Password

>>>
```

Section 5: Compile Once, Validate Many

```
import re

# Compile the email pattern once
email_pattern = re.compile(
    r'^[A-Za-z0-9]+[A-Za-z0-9._%+-]*@[A-Za-z0-9.-]+\.(com|org|edu|in)$'
)

# Sample list of email IDs
emails = [
    "john.doe@gmail.com",
    "alice_123@yahoo.com",
    "student@college.edu",
    "user@test.in",
    "invalidemail.com",
    "abc@domain",
    "user@site.org"
]

# Validate each email
for email in emails:
    if email_pattern.fullmatch(email):
        print(email, "-> Valid")
    else:
        print(email, "-> Invalid")
```

```
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr  2 2024, 10:12:12) [MSC v.1938 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
== RESTART: C:/Users/Abinesh/AppData/Local/Programs/Python/Python311/at1.3.py ==
john.doe@gmail.com -> Valid
alice_123@yahoo.com -> Valid
student@college.edu -> Valid
user@test.in -> Valid
invalidemail.com -> Invalid
abc@domain -> Invalid
user@site.org -> Valid
>>>
```